

Arrays og referencer



Program

- Array og referencer
- Øvelser
- Opsamling

Arrays

Diskuter med din sidemakker: *Hvorfor bruger vi variable?*

Indtil nu har vi arbejdet med variable, som gemmer **en enkelt værdi**.

```
int coins;  
coins = 50;
```

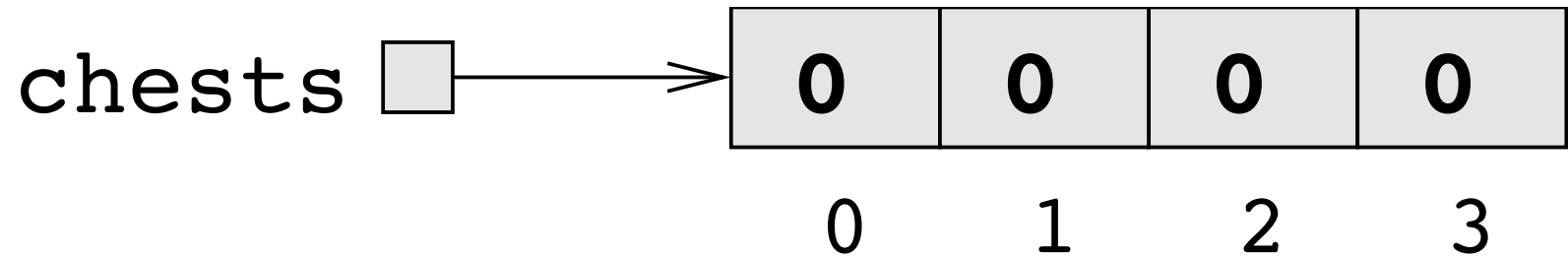
Nogle gange har vi brug for at gemme **flere værdier** i én variabel



```
int[] chests;
```

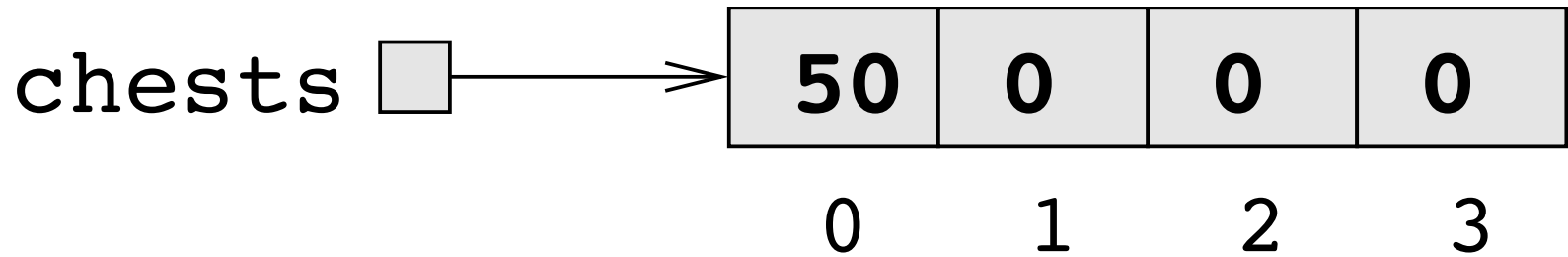


```
int[] chests = new int[4];
```



Piraten gemmer 50 mønter i den **første** kiste.

```
int[] chests = new int[4];  
chests[0] = 50;
```

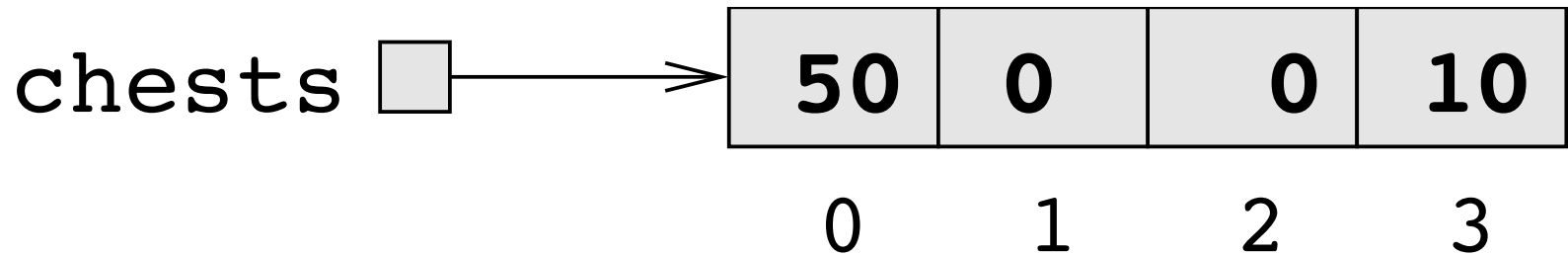


```
int[] chests = new int[4];  
chests[0] = 50;
```

Lad os gemme 10 mønter i den sidste kiste.

```
chests[??] = 10;
```

- hvad er indekset for den sidste kiste?



```
int[] chests = new int[4];  
chests[0] = 50;  
chests[3] = 10;
```

Vi kan initialisere en array med værdier direkte:

```
int[] chests = {50, 0, 0, 10};
```

Vi kan tilgå værdierne i array'et ved at bruge indekset.

Spørgsmål: *Hvilket indeks har den anden kiste?*


```
int[] chests = {50, 0, 0, 10};  
int secondChest = chests[1]; // 0
```

Vi kan fjerne 20 mønter fra den første kiste.

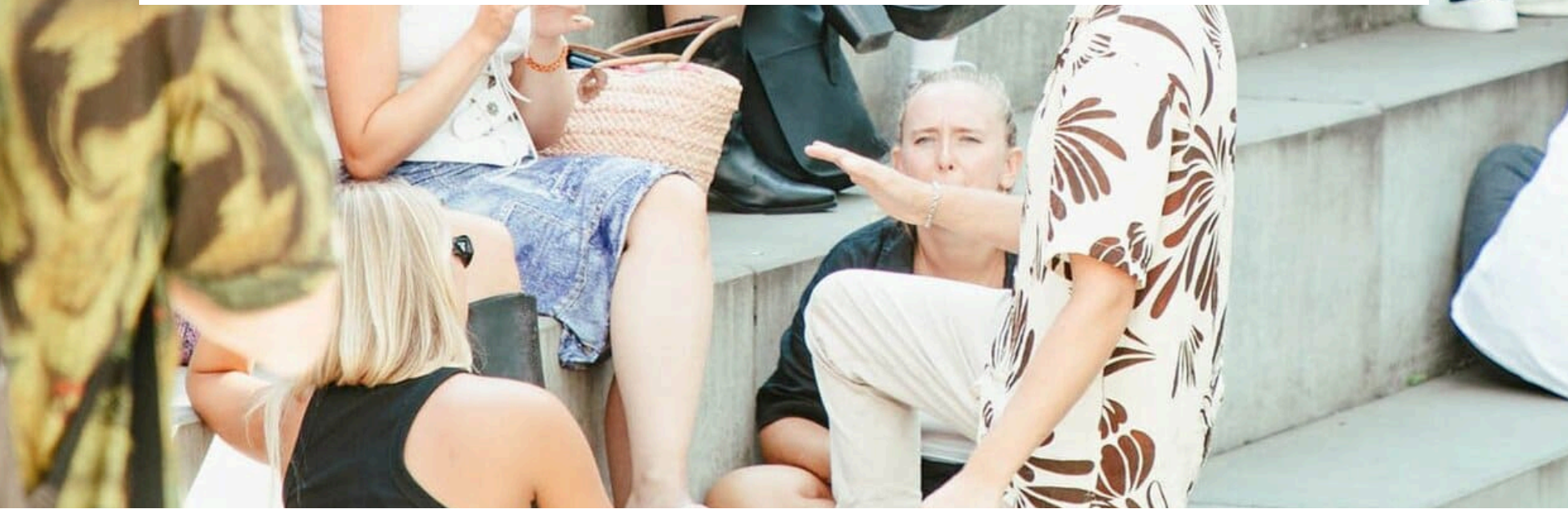
```
chests[0] = chests[0] - 20; // 30
```

Alternativt, med -=

```
chests[0] -= 20; // 30
```



Primitive vs. reference



Primitive datatyper (fx `int`, `double`, `char`)

```
int a = 5;  
int b = a;  
a = 10;  
System.out.println(b); // Hvad bliver der printet?
```

Reference datatyper (fx **String**, arrays, objekter)

```
int[] arr1 = {1, 2, 3};  
int[] arr2 = arr1;  
arr1[0] = 10;  
System.out.println(arr2[0]); // Hvad bliver der printet?
```

Kort sagt:

- reference datatyper **gemmer en reference** til data, mens
- primitive datatyper gemmer **selve dataen**.

Stack og heap

Java bruger to hukommelsesområder til at gemme data: stack og heap.

- **Stack**: Her gemmes **primitive datatyper og referencer**.
- **Heap**: Her gemmes den data, som referencer peger på.

- **Stack'en** er hurtig at tilgå, men har **begrænset størrelse**
- **Heap'en** er langsommere at tilgå, men har **større kapacitet**

Stack Frame:

```
+-----+  
| name   = 0x1A2B ----->  
| image  = 0x3C4D ----->  
| height = 179.50 |  
| age    = 34      |  
+-----+
```

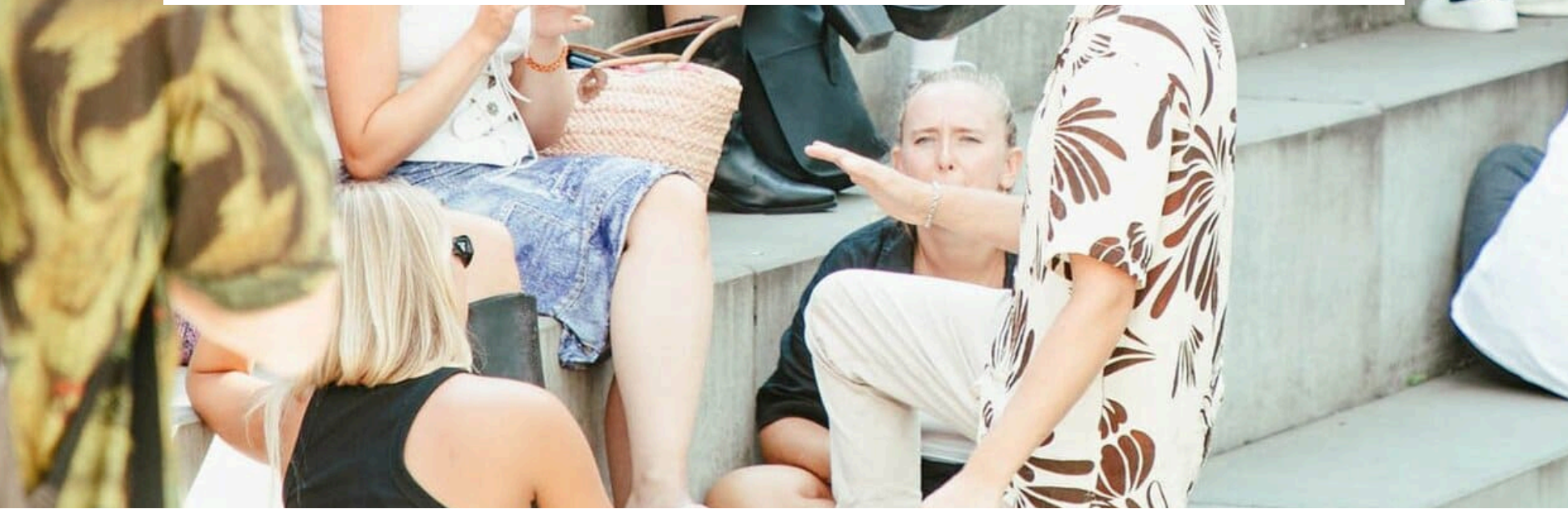
Heap:

```
+-----+  
| 0x1A2B: "Jakob" |  
| 0x3C4D: [0, 1, 255, 3] |  
+-----+
```

0x1A2B og 0x3C4D er referecer (adresser) til data i heap'en.



Arrays og indeksering



```
String[] names = {"Alice", "Bob", "Charlie", "Diana"};
```

hvorfor er **Alice** i **names[0]** og ikke i **names[1]**?

Hvorfor starter vi ikke på 1?

Indekset er i virkeligheden en **forskydning** (offset) fra hukommelsesadressen, så

- det første element er ved **forskydning 0**,
- det andet ved **forskydning 1**, osv.

A young woman with long, wavy blonde hair is smiling and looking off to the side. She is wearing a blue and white horizontally striped sweater. The background is a blurred outdoor setting with a brick building and some greenery.

call-by-value

I Java er alle argumenter til metoder **call-by-value**.

Det betyder at:

- primitive datatyper bliver kopieret (call-by-value)
- reference datatyper bliver kopieret (call-by-value af referencen)

```
public static void modify(int a) {  
    a = 100;  
}
```

```
int x = 5;  
modify(x);
```

```
System.out.println(x); // Hvad bliver der printet?
```

```
public static void modify(int[] arr) {  
    arr[0] = 100;  
}
```

```
int[] chests = {50, 0, 0, 10};  
modify(chests);
```

```
System.out.println(chests[0]); // Hvad bliver der printet?
```


A group of people are gathered around a table in a meeting. A man in a white t-shirt is on the left, looking towards the right. A woman with blonde hair in a dark blazer is on the right, smiling and looking at the camera. In the foreground, a laptop screen displays a colorful dashboard with various charts and graphs. A white cup and a silver thermos are also on the table. The text "for og foreach" is overlaid in the center in a white serif font.

for og foreach

Vi kan bruge en **for**-løkke til at iterere over et array:

```
int[] chests = {50, 0, 0, 10};  
  
for (int i = 0; i < chests.length; i++) {  
    System.out.println(chests[i]);  
}
```

... men der er en smartere måde:

Vi kan bruge denne **foreach**-løkke til at iterere over et array:

```
for (int chest : chests) {  
    System.out.println(chest);  
}
```

Hvilke tre ting tager du med fra i dag?

